1 Using the Repository Client as Development Tool

The Repository Client is not only used as service documentation. You can also use it to edit the repository data to create new services.

1.1 How to create new contents

The data structure of the repository is hierarchical. This means that a service is always part of a domain, a service parameter is always part of a service and properties are always the children of a domain.

This again means that you always work in a "top-down" view when working in the repository. For example, if you want to create a new service, you must ensure that the domain where you want to create the service does actually exist. If you want to create a service parameter for a new service, this one should also exist at this time, etc.

If you keep this basic information in mind and design your workflow on basis of this structure, you will spare a lot of unnecessary work and frustration.

ŸIf you want to make changes in the repository, it is recommended to create another domain set within the work set to manage your modifications. ŸDo not forget to check the “IsWriteable” option – otherwise you will not be able to save any changes later on.

Please also note that the workset is not part of the repository data model and changes in the workset will only become effective upon re-loading the repository. We therefore recommend that the workset is defined for the imminent task, first.

Prior to starting your work, it is reasonable to make yourself familiar with the **Error! Reference source not found.**.

Example: Creating new services

The use case in the following illustrates the workflows that are involved in the generation of services.

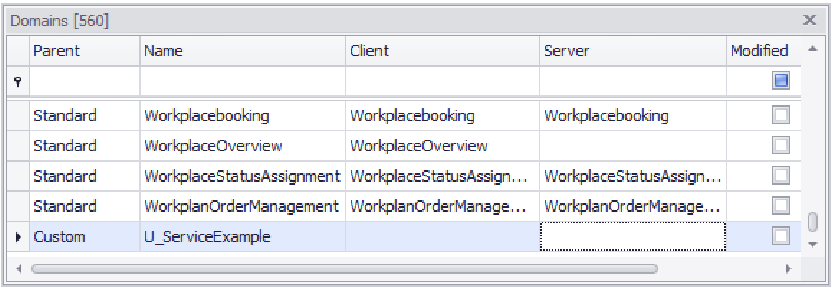
1. Import the latest repository version.
2. Open the repository client.
3. Create a new **Error! Reference source not found.** with two domain sets (standard, custom).

Workset [ServiceExample.work]

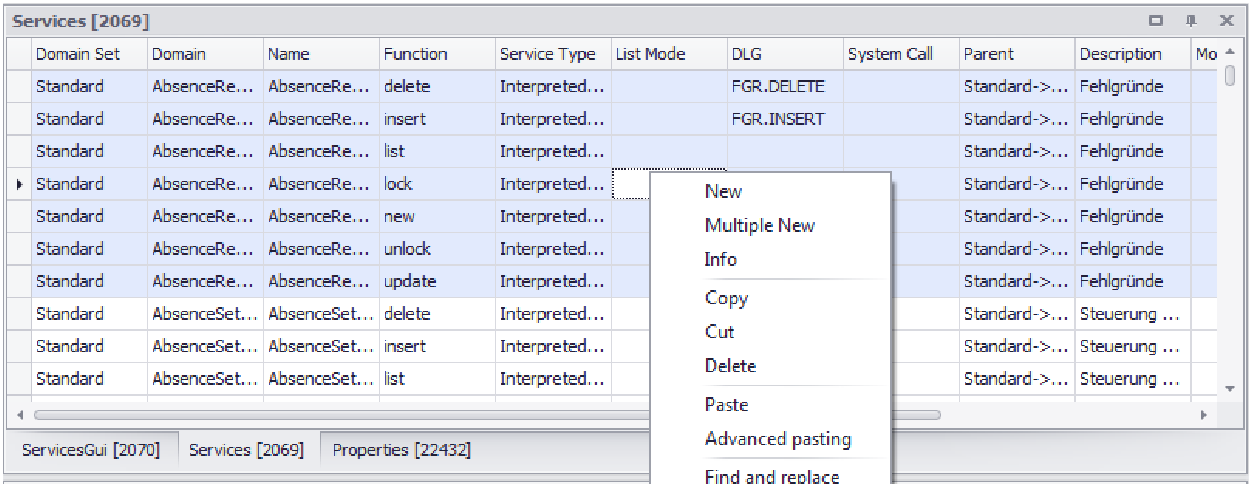
Name	Client Source	Server Source	Overrides	Priority	Is Writeable	Modified
Standard	C:\StdDoms\ClientDomain.zip	C:\StdDoms\ServerDomain.zip		0	☐	☐
Custom	C:\CustDoms\ServiceExample\Client	C:\CustDoms\ServiceExample\Server	Standard	0	☑	☐

4. Save the new workset.

5. Load the repository.
6. Create a new domain via the context menu (right click the domain view→New) and edit the domain data (Name = "U_ServiceExample").

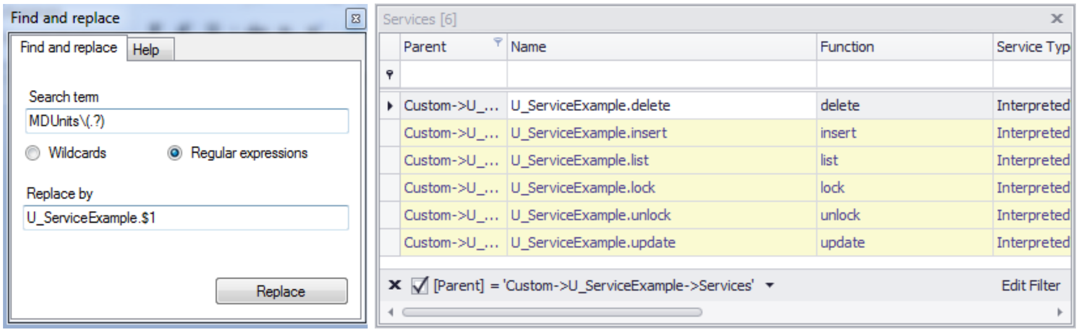


7. Copy other services to be used as model via the context menu:



8. Select the U_ServiceExample domain in the domain view. The selected domain becomes the active domain which is used to filter the services. (This is only possible with an active relation from domain to service).
9. The services can be inserted using the context menu in the service view. The active filter (set before) defines which of the copied services can be added to the new domain. All included service parameters are automatically copied at the same time.
   Of course you can also use proven key combinations, e.g. Ctrl+C for copying, Ctrl+X for cutting, as well as Ctrl+V for pasting/inserting.

10. At this point, an adjustment of the service names is required. For example, you can change the names using the Find and Replace dialog that is also available via the context menu:



11. Adjust / remove / add service parameters in known manner.
12. Save.

    The files have been written to the specified location in the hard disk.
13. *Optional:* Export

    You can directly write into a structure, which you can use to directly test your changes.

This example could well have been extended. But to directly start work with the client, the example used illustrates the major steps to get a first idea.

At this point, you might ask how you have to proceed with the GUI part of the services. Of course you could also copy them into the new domain and change them. Other option: right-click the domain to create the GUI part of the services automatically and to add potentially missing properties from the created services.

It might be easier to copy the complete domain and to simply delete the elements that are not required.

One level further down, the properties of the structure become even more evident. If only a few service parameters of a service are required for a new one, it might be easier to copy the complete service and to delete the excessive parameters. This spares the entire "Creation" of a new service.

1.2 Context menu of the table view/grid

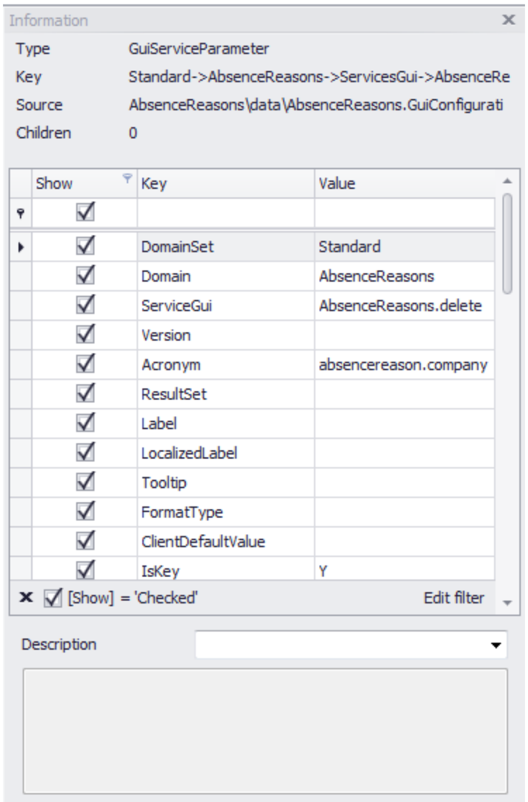
A context menu opens if you right-click the tables. The menu includes different entries depending on the type and status of the table view.

New

Use this function to add a new row to the table view. In the columns with set filters, the values will be set according to the filters in the new row. If, for example, a filter is set to "LIKE Test%" or "= Test", the cell value is set to "Test". Advanced filters are not supported.

Info

Click *Info* to open an InfoPanel. The panel is bound to the source table and shows information on the selected data record in the source table. In addition to a clear identification of the data record, the data source from which it was loaded is shown. The entry *Children* shows the number of data records allocated. In the given example, the service parameter has 2 children service parameters. In addition, the service attribute values are listed in a table. In the bottom area of the dialog, the description stored for the data record is shown.



Copy

Deposits selected rows into the clipboard. This option is only offered if the view contains data.

Cut

Deposits selected rows into the clipboard and subsequently deletes them. This option is only offered if the view contains data.

Delete

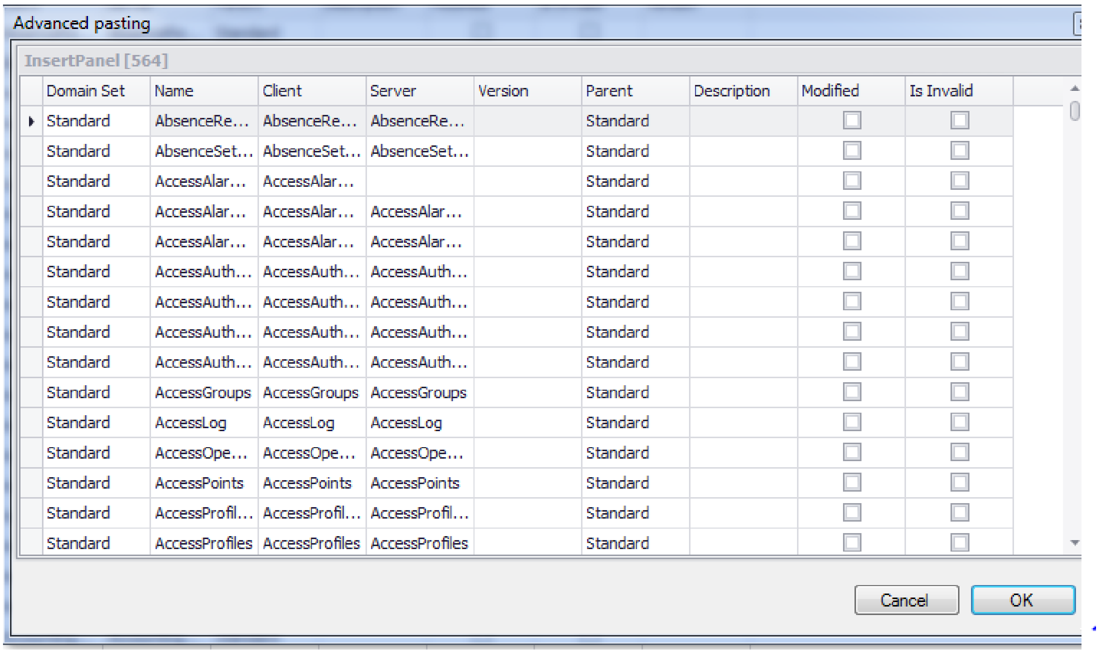
If you select this function, a dialog listing the data records to be deleted is shown. Click "Yes" to delete them; "No" will cancel the deletion process.

Insert

This function adds rows from the clipboard to the grid. This option is only offered if the cache/clipboard contains data which may be inserted in the currently shown table.

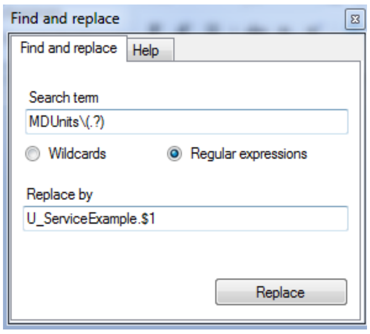
Advanced pasting

Contrary to 'Insert', this function opens a dialog that allows to edit the entries in the cache/clipboard before you insert them. It is possible to allocate new values to individual cells and to all cells of a column. You can cancel the *Insert* process. You can only select this option if the cache/clipboard contains data which may be inserted in the currently shown table.



Find and replace

Use this function to find and replace values within a column. If you select this function, the following dialog opens:



Specify the search term and the term that should replace the search term and confirm by "Replace". For example, use this dialog to replace prefixes. In addition, this function supports regular terms.

Relations

This entry leads to a list with identifiers of relations that are defined in the relations table. If you select one of these identifiers, a list of the currently shown tables opens. Select one of these tables to instantiate this relation. A new relation of this type is automatically created; the source is set on the current table and the target on the selected table, respectively. For details on the semantics of relations, please refer to section **Error! Reference source not found.**.

Show reference

You can use this function to open a table view listing the data to which the entry of the current cell refers. For details on the semantics of references, please refer to section **Error! Reference source not found.**.

Get references

This entry opens a new table which will fill the current data record with values from the referenced data records. For details on the semantics of this function, please refer to section **Error! Reference source not found.**.

Create GUI configuration

This entry is only shown in the context menu of the domain panel. Use this function to create the ServicesGUI according to the services of the selected domain.

Create properties

Also this entry is only available in the domain panel. Use this entry to create properties according to the ServiceParameterGui.

Create service based on SQL

Also this entry is only available in the domain panel. You can use this function to generate services. You use an SQL statement to extract information on fields and tables.

- A "select" statement generates a service of type InterpretedJavaService.
- A "create table" statement generates services of type InterpretedBapiService to edit data and a list service of type InterpretedJavaService to show the respective data.

The information included in existing parameters in the repository is added to the information on the individual fields, if possible (the allocation is based on the table and the field name).

Note: This function only helps to create services. It is up to the user to ensure the correctness.

Example 1 ("select" statement)

```
select m.masch_nr,
       m.bez_lang,
       k.bezeichnung,
       k.sap_logical_system
from   maschinen m
       left outer join kostenstellen k
                    on k.kostenstelle = m.kostenstelle
```

This statement is used to generate a list service. No existing acronym for the column `k.sap_logical_system` can be found. For this reason, the column is marked in the "acronym" with "<TODO>" and the acronym must be defined manually (delete <TODO>). Then you can run the list service.

Example 2 ("create table" statement)

```
create table u_test_table
  (
     test_string  char(20),
     test_date    date,
     test_integer integer,
     test_decimal decimal(18, 6),
     test_serial  serial
  );
```

This statement requires more manual rework:

- Check acronyms
- If you use the database type "serial": with database type "serial", you must assign WebServiceType=integer. For the services delete, lock, unlock and update, you must include the constraint "SERIAL|" in the serial column and specify it as mandatory parameter (IsMandatory=Y). For the service insert, make the settings IsMandatory=N, IsResult=Y and IsSpecialParameter=N.
- For the editing services, you can define key columns (except for serials) if required (Constraint "KEY=n|") and define them as mandatory parameters (IsMandatory=Y)
- The services delete, lock and unlock only require the key columns (constraint "KEY=n" or "SERIAL|"). Delete the columns that are not used.
- Check WebServiceType with all service parameters and complete, if required.
- Also respect the further notes on the generation of services in this document.

Then you can run the service.

Operator Assistant

If a service has a lot of parameters, it is a complex task to set the columns for the operators supported by the service parameter in the table view of the service parameters. To facilitate this task, an assistant exists to manage the suppported operators.

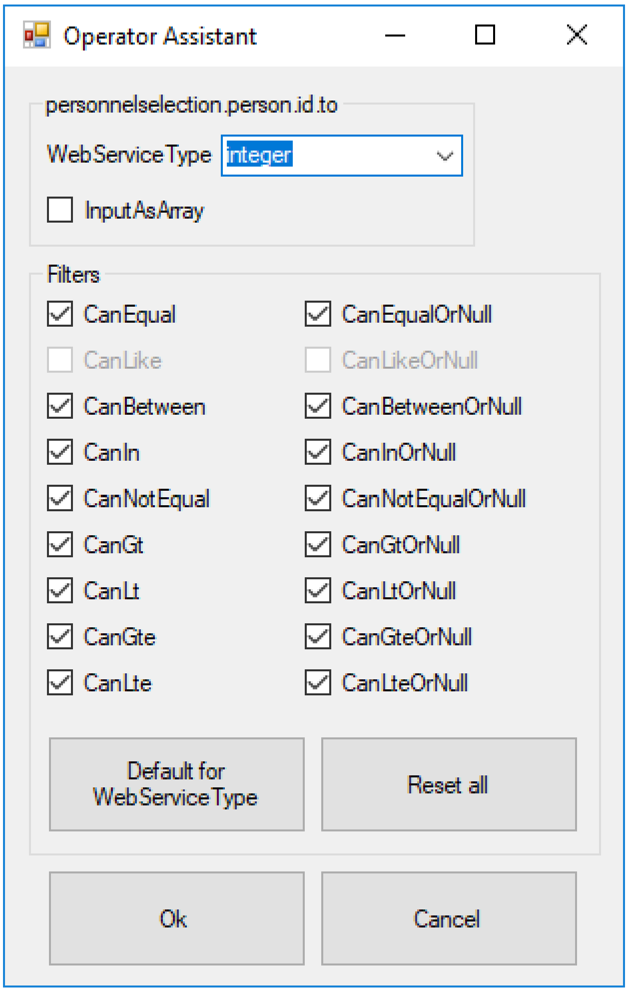
Start an *Operator Assistant* in the context menu of the *ServiceParameters* view.

The *Operator Assistant* is available in the MPDV Repository Client as of version 1.8.STD.66280.

You can also copy the supported operators from one service parameter to another in one operation and use the function *Find and replace* for all operators.

Operator Assistant

To start the *Operator Assistant*, select a row in the view ServiceParameters and right-click to open the context menu. Select the entry *Operator Assistant*.



In this dialog, you can edit all operators of the service parameter and use the option *InputAsArray*. If you click the OK button, the settings are applied to the columns in the table view of the service parameter.

Button *Default for WebServiceType*

The options are predefined according to the WebServiceType.

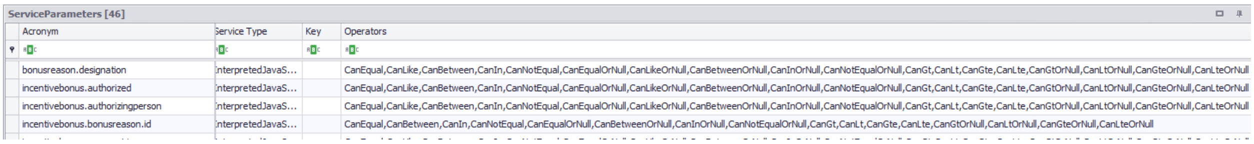
Button *Reset all*

All options are deactivated.

Copying operators from one ServiceParameter to another

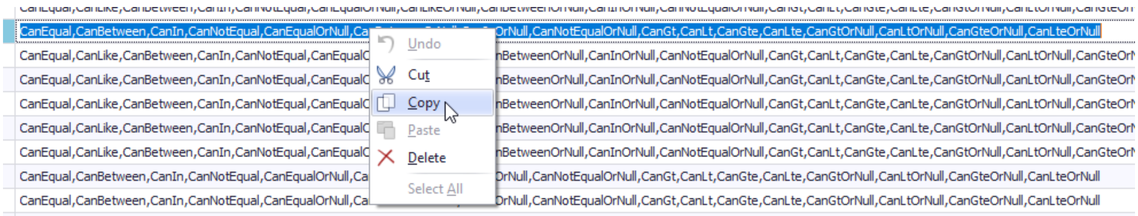
The *Operator Assistant* is available in the MPDV Repository Client as of version 1.8.STD.66280.

The table view of the ServiceParameters provides a column *Operators*. You can use this column to manage the options using the *Operator Assistant* and to manually edit the operators of a ServiceParameter in a single column of the table. Changes to the column *Operators* and to the different columns *CanEqual*, *CanLike,...* are automatically synchronized.



Proceed as follows to copy the operator options from one row to another in one operation:

- Select the column *Operators* of the service parameter that includes the operators you want to copy and double-click. The text in the column is then selected.
- Copy the text to the clipboard:

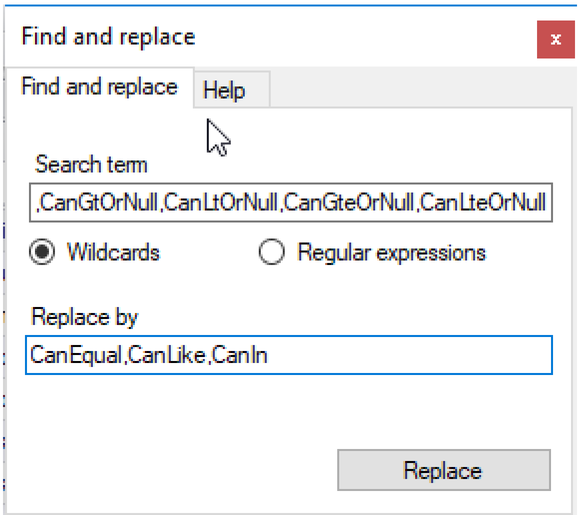


- Select the column *Operators* of the service parameter to which you want to copy the operators and double-click. The text in the column is then selected.
- Paste the text from the clipboard.
- Press the RETURN key.

Replacing operators using *Find and replace*

This function is available for the column *Operators.* You proceed in a similar way like described in the section above:

- Select the column *Operators* of the service parameter that includes the operators you want to replace by another combination.
- Select *Find and replace* in the context menu.
- The dialog *Find and replace* opens.



The value of the previously selected service parameter is preassigned in the *search term* field.

- Enter the combination that replaces the search term.
- Confirm by clicking the button *Replace*. The assistant replaces the operator combination specified in the search term by the new combination in all service parameters in the table. The change is also performed in the columns of the separate operators "CanEqual", "CanLike",...

1.3 Export

Use the application menu (*Repository* → *Export repository*) to activate the export dialog. Here, you can make a detailed selection of the data records that you want to export. Settings in this dialog are displayed again when re-opening the dialog.

In the "Domain filter" area you can specify the domain set that you want to export. If you do not make any entry here, all domain sets are exported. In addition, you can set a filter for the domains that you want to export. If you do not set any filter, all domains of the relevant domain set are exported. In the "File filter" area, you can specify which data types are to be exported.

In the "Export paths" area, you can store and select up to three paths for export. For each path, you can specify the export structure that you want to use.

- **Client Domain:** Data in this structure can be read by the Repository Client.
- **Server Domain:** Data in this structure can be read by the Repository Client.
- **Client Runtime:** Data in this structure can be read and processed by the client.
- **Server runtime:** Data in this structure can be read and processed by the server.

Start the data export via "Export". When the export is completed, a dialog opens showing the number of exported data records.

1.4 Validation

The Repository Client provides an integrated validation function checking the syntax of the columns (or property) to be edited and the syntax of a data record itself. The validation function also checks the consistency between several data records (data types), in particular master-detail relations of individual domains, and it provides a multiple validation and a validation subject to a function type or data type (e.g. Service --> ServiceType). This function is performed when you edit data or when you click the validation button.